Paper Review: DAAIP: Deadblock Aware Adaptive Insertion Policy for High Performance Caching

Vedang Kashyap
Department of Electronics Engineering (VLSI Design)
Vellore Institute of Technology, Vellore, India

I. Introduction

RRIP-based policies like SRRIP and ABRip predict a re-reference interval per block but cannot distinguish a cache-friendly application from a streaming one sharing the same LLC, so streaming blocks get the same long re-reference treatment as cache-friendly blocks in the same cache set.

A. Contribution

- Shows that a large share of blocks inserted into the LLC, on average 85%, are never re-referenced before eviction, and that this deadblock percentage stays roughly constant over consecutive phases of an application.
- Proposes DAAIP, which uses per-core deadblock feedback from evicted lines to adaptively choose an insertion position between LRU and one position above LRU.

II. Proposal

DAAIP tracks each application's deadblock behavior at runtime to decide where its incoming blocks are inserted. Each block carries a one-bit reuse flag, zero on insertion and set if re-referenced before eviction, plus bits identifying the installing core. Per core, two 16-bit counters track total blocks installed and deadblocks evicted. Every 2^{16} misses, the phase's deadblock percentage is compared to a threshold: above it, new blocks go to LRU; otherwise, one position above LRU. Both counters halve at each boundary, weighting recent behavior while keeping some history. Blocks promote to MRU only on re-reference; the scheme replaces SRRIP's insertion step directly.

III. Key Results Achieved

- DAAIP improves dual-core system throughput over SRRIP by up to 21% and 5.8% on average across 39 multiprogrammed workload mixes.
- DAAIP reduces MPKI by up to 22% on evaluated SPEC CPU2006 benchmarks compared to SRRIP.
- DAAIP outperforms ABRip, a prior application-aware RRIP policy, by 4.6% on average on system throughput, and unlike ABRip, improves performance for both Cf-Cf and Cf-Str workload mixes.
- Total hardware overhead is approximately 16KB, about 0.0039% of a 4MB LLC.

IV. Key Takeaway

1) Feedback from what actually happened to a block, whether it was ever reused before eviction, is a more direct learning signal than any prediction made only at insertion time, since the outcome itself becomes the ground truth the next decision can be tuned against.
2) Application-level behavior, not just per-set or per-block behavior, is a meaningful unit to reason about in a shared cache; treating a core's overall reuse tendency as the signal, rather than only local per-set counters as in ABRip, lets one region's behavior inform decisions for the whole core.
3) Coarse-grained, low-cost hardware, a single reuse bit and two small counters per core, can capture enough signal to adaptively partition a shared cache without the cost of maintaining full per-application recency state or auxiliary tag directories.

V. Weakness

- DAAIP only chooses between two insertion positions, LRU and one position above LRU, unlike RRIP-style policies that use multiple RRPV levels, so its adaptivity is coarser than the granularity RRIP itself already offers.
- The prose only says counters are halved to retain past behavior, without stating that this halving happens exactly at the phase boundary, so the next phase starts from half the prior evidence rather than zero; this is only clear from Algorithm 1, not the text.
- The paper reports that DAAIP loses performance compared to SRRIP on a few workload mixes, such as gcc-milc against ABRip, but does not explain why the phase-based threshold fails for these specific cases.
- Evaluation is limited to dual-core systems and Cf-Cf or Cf-Str mixes; Str-Str mixes are explicitly excluded and quad-core scalability is left as future work.

VI. Extension

- The paper's own future work names using auxiliary tag directories to dynamically switch between the best-performing policy per workload mix, since DAAIP sometimes underperforms SRRIP, and extending the scheme to quad-core systems.
- DAAIP measures deadblock percentage per core as a whole, but an unexplored idea is measuring it per cache set or per region of the address space within a core, so a core running one cache-friendly and one streaming code region internally could still get differentiated insertion decisions rather than one blended decision for the entire core.
- The phase length is a fixed count of misses decided once at design time; an untried alternative is making the phase length itself adaptive, shortening it when the deadblock percentage is changing rapidly between consecutive phases and lengthening it when the percentage is stable, so the counters halve at a rate matched to how quickly the application's behavior is actually drifting.
- DAAIP's only two insertion positions are LRU and one step above LRU; combining its deadblock feedback with RRIP's multi-level RRPV scale is unexplored and could yield a graded value instead of two fixed points.

VII. Reference

Newton, Sujit Kr Mahto, Suhit Pai, and Virendra Singh. *DAAIP: Deadblock Aware Adaptive Insertion Policy for High Performance Caching*. In Proceedings of the 2017 IEEE 35th International Conference on Computer Design (ICCD), 2017.